TUTORIAL PRÁTICO · WINDOWS 11

Instalar, executar e testar o projeto G10

Docker Desktop + Java Spring Boot + PostgreSQL + PgBouncer: guia passo a passo, mesmo para quem ainda não usou Docker.

> Regra de ouro: para executar o projeto completo você precisa instalar somente o Docker Desktop. Java, Maven e PostgreSQL são fornecidos dentro dos contêineres. Para programar Java diretamente no Windows, instalar JDK 21 e Maven é opcional.

O que já vem no projeto

SERVIÇO	TECNOLOGIA	RESPONSABILIDADE
frontend	HTML/CSS + JavaScript + Nginx	Telas e proxy reverso /api
backend	Java 21 + Spring Boot 3.5.16	API, validação, JWT, JDBC
pgbouncer	PgBouncer (transaction pooling)	Pool de conexões PostgreSQL
db	PostgreSQL 16	Tabelas, roles, RLS, funções e triggers

Credenciais para o laboratório

PERFIL	LOGIN	SENHA
Coordenação	coordenacao	Coord@123
Professora Ana	ana	Prof@123
Professor Bruno	bruno	Prof@123

Estas credenciais são de demonstração e não devem ser usadas em produção.

Etapa 1 — Preparar Docker no Windows 11

- Abra o PowerShell e digite wsl --status. Se o WSL não estiver instalado, execute wsl --install em terminal de administrador; reinicie caso solicitado.
- Instale Docker Desktop pelo site oficial docker.com/products/docker-desktop. Configure o backend WSL 2 e mantenha Linux containers.
- Abra Docker Desktop e aguarde o motor Docker iniciar. No PowerShell, confira os comandos abaixo.

```
docker --version
docker compose version
docker info
```

Etapa 2 — Entrar na pasta do projeto

- Extraia o arquivo ZIP. Entre na pasta G10-Gestao-Academica que contém docker-compose.yml.
- No Explorador de Arquivos, digite powershell na barra de endereço e tecle Enter.
- Digite dir; confira a presença das pastas backend, frontend, bd, pgbouncer.

Etapa 3 — Subir os contêineres

```
docker compose up --build
```

O primeiro uso precisa de internet para baixar as imagens e bibliotecas Java. O Docker compila o Spring Boot e inicializa o PostgreSQL automaticamente.

Quando os serviços iniciarem, abra no navegador: http://localhost:8080.

```
docker compose ps
docker compose logs -f backend
```

Esperado: serviços db, pgbouncer e backend saudáveis, frontend em execução. Não feche o terminal principal enquanto usar up sem -d.

Etapa 4 — Testar as funções do sistema

- Faça login como coordenacao / Coord@123 e abra a tela Alunos. Cadastre, edite e confirme a alteração.
- Abra Notas e boletim e altere duas notas. A API Java chama a procedure sp_lancar_notas; a function recalcula a média.
- Abra Auditoria. Confira o evento UPDATE registrado por trigger com valores de antes e depois.
- Entre como ana e bruno para comparar quais matrículas cada perfil pode ver.
- Abra PostgreSQL / RLS como coordenador e execute o teste de timeout de 3 segundos.

Etapa 5 — Rodar os testes de banco do professor

```
.\scripts\verificar-requisitos.ps1
.\scripts\testar-rls.ps1
.\scripts\consultar-pool.ps1
```

Resultados esperados: o primeiro mostra schemas, roles e objetos SQL; o segundo mostra Ana = 3 matrículas, Bruno = 2, coordenação = 5 e impede escrita indevida; o terceiro exibe SHOW POOLS do PgBouncer.

Se o PowerShell bloquear scripts, use esta forma (somente para o processo chamado):

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\testar-rls.ps1
```

Comandos mais importantes do Docker

COMANDO	EFEITO
docker compose up -d --build	Compila e sobe tudo em segundo plano.
docker compose ps	Lista contêineres e estado.
docker compose logs -f backend	Mostra erros do Spring Boot.
docker compose down	Para tudo, mantendo o volume do banco.
docker compose down -v	PERIGO: apaga o volume e os dados.

Para entender onde fica o Java

- Application.java: inicia Spring Boot.
- api/AcademicoController.java: métodos REST GET/POST/PUT/DELETE, notas e frequência.
- api/AuthController.java: login dos usuários cadastrados.
- config/DataSources.java: três pools HikariCP para acesso SQL.
- config/DatabaseExecutor.java: transação e identidade de usuário para a RLS.
- security/: assinatura/validação do token JWT e filtro de autenticação.

> Fluxo: navegador → HTTP /api → controller Spring Boot → SQL parametrizado (JdbcTemplate) → PgBouncer → PostgreSQL → JSON de resposta → tela atualizada.

Desenvolver Java fora do Docker (opcional)

Instale JDK 21 e Maven 3.9+ e configure o PATH. Use java -version e mvn -version para confirmar. Para compilar/testar sem executar a aplicação:

```
cd backend
mvn test
mvn package
```

Para executar a API localmente, primeiro exponha o PgBouncer em localhost com o arquivo docker-compose.dev.yml e depois inicie o Spring (não inicie o frontend Docker no mesmo momento, pois ele usa a porta 8080):

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d db pgbouncer
$env:DB_JDBC_URL='jdbc:postgresql://localhost:6432/academico?prepareThreshold=0'
cd backend
mvn spring-boot:run
```

Se der erro

- Docker daemon inacessível: abra Docker Desktop e espere iniciar.
- Erro do WSL: rode wsl --status; verifique virtualização na BIOS e reinicie.
- Porta 8080 ocupada: altere 127.0.0.1:8080:80 para 127.0.0.1:8081:80 no frontend e acesse :8081.
- Mudou backup.sql sem efeito: SQL inicial roda apenas na criação do volume; down -v apaga todos os dados.
- Contêiner não sobe: execute docker compose logs -f db pgbouncer backend frontend.

Tutorial completo e publicação GitHub: docs/como-rodar-windows.md e README.md.